

POLYMULT 1.4

Fast Polynomial Multiplication

Anton S. Mosunov

June 14, 2026

Contents

1	Introduction	1
2	Changes since the previous version	2
3	Dependencies in POLYMULT	4
4	Building and using POLYMULT	4
5	Reporting issues	4
6	Files	4
7	Supported series	5
8	File types	5
9	Initialization	6
10	Out-of-core operations on polynomials	7
11	Utilizing the executable	8
12	Examples	9
13	Utilizing the library	10

1 Introduction

The POLYMULT library provides subroutines for out-of-core Fast-Fourier-Transform (FFT) based polynomial multiplication using `OpenMP` for parallelization. The “out-of-core” here means that the resulting

files and the intermediate computations get stored into the hard disk. The library is designed for multi-core and multiprocessor environments.

The library is designed specifically for very large polynomials with non-negative integer coefficients, possibly exceeding 2^{37} in their degree. Further in this manual, we use the words “series” and “polynomial” interchangeably, as all the polynomials that are available in POLYMULT get initialized from specific infinite series.

The out-of-core FFT-based polynomial multiplication technique is due to Hart, Tornara and Watkins [HTW10]. The detailed description of this technique can also be found in [Mos14, Section 4]. In simple terms, the problem of multiplication of two large polynomials gets reduced to several multiplications of smaller polynomials over finite fields. The multiplication of smaller polynomials (in our experiments, of degree less than 2^{25}) is performed using the subroutines implemented in the FLINT library.

POLYMULT is maintained by Anton Mosunov, Cornell University, and is an appendix to the Master’s thesis [Mos14]. It is highly recommended to read the thesis (Section 4 in particular) before utilizing the library, as certain notions, such as the bundling parameter or the bit size parameter, are not defined in this manual.

The manual for POLYMULT is based on the LATEX template for the manual of FLINT 1.0:

<http://web.mit.edu/sage/export/tmp/flint-1.1/doc/flint-roadmap.tex>. The contents of source files as well as header files are organized FLINT 2.5.2.

2 Changes since the previous version

June 14th, 2026. The following changes had been made in the version 1.4:

- There is no more `Makefile`, as it doesn’t work. To add it is a future (likely very distant future) project. To compile POLYMULT, use the following command:

```
gcc -fopenmp -lflint main.c init.c mult.c -o polymult
```

- In the file `init.c`, fixed a bug in the subroutine `void init_block_nabla_product(...)` which calculates the coefficients of $\prod_{i=1}^2 c_i q^{r_i} \nabla_{a_i, m_i}(q^{s_i})$ for $\min \leq k < \text{size} + \min$.
- Included a bash script `tabulate_class_numbers.sh` that tabulates class numbers using the formulas from both [Mos14] and [BK22]. **If you don’t want to think hard about how to tabulate class numbers, all you need is to run this script.** Modify the parameters in the script according to your system’s configurations. In order for the script to work properly, make sure that the value of `discriminant_bound` is divisible by $15 \cdot 2^{27}$.
- POLYMULT can now multiply many more series. However, the functionality for dividing one series by the other has been removed, because it was far too impractical.
- Added a collection of basic C programs that enable one to post-process class number files resulting from polynomial multiplication. These programs are located in the folder `post-processing` and, with the exception of `check`, are utilized by `tabulate_class_numbers.sh`. They are
 - `check`: uses PARI to check if the class numbers $h(\Delta)$ with fundamental discriminants $\Delta < 0$ satisfying $|\Delta| \equiv a \pmod{m}$ recorded in a given file are correct. In the case when $|\Delta| \equiv 71, 119 \pmod{120}$, checks if the recorded class number is 0. In order for the program to work correctly, file size must be divisible by `BUFSIZE= 65536`. The program `check` can be compiled with

```
clang -lpari check.c -o check
```

The usage is

```
./check [file_with_class_numbers] [first_discriminant] [a] [m]
```

- **divide**: Takes as an input binary files containing unsigned integers and produces new files with these unsigned integers divided by a fixed positive number. Notice that all files must store **unsigned int** values in binary, have exactly the same nonzero size, and this size must be divisible by `BUFSIZE= 65536`. The program **divide** can be compiled with

```
clang -fopenmp divide.c -o divide
```

The usage is

```
./divide [number_of_files] [output_file_prefix] [input_file_prefix] [number_to_divide_by]
```

- **merge**: Merges data containing class numbers for fundamental discriminants with $|\Delta| \equiv a_1, a_2, \dots, a_k \pmod{m_{\text{big}}}$ into a single file with $|\Delta| \equiv a_1 \pmod{m_{\text{small}}}$. In order for the program to work properly, must have $0 \leq a_1 < m_{\text{small}}$ and $m_{\text{small}} \mid m_{\text{big}}$. As long as this is the case, we have $a_1, a_2 = a_1 + m_{\text{small}}, a_3 = a_1 + 2m_{\text{small}}, \dots$, and so on. If a file for a certain congruence class is missing, zero is recorded in place of the actual class number. In order for the program to work properly all existing files must have exactly the same nonzero size, and this size must be divisible by `BUFSIZE= 65536`. The program **merge** can be compiled with

```
clang -fopenmp merge.c -o merge
```

The usage is

```
./merge [path] [number_of_files] [a] [small_m] [big_m]
```

- **replace**: This program takes as an input an index n , an unsigned integer k , and a binary file containing unsigned integers, and replaces the n -th entry in the binary file with k .

```
clang replace.c -o replace
```

The usage is

```
./replace [filename] [index] [new_value]
```

- **replace**: This program takes as an input prefixes two or more input binary files, sums their contents and saves them into an output file.

EXAMPLE: `./sum 32 ~/res ~/a ~/b`

In this example, first input files have names `a0, a1, ..., a31`, while second input files have names `b0, b1, ..., b31`. The resulting files are called `res0, res1, ..., res31`. If we view each file as a huge vector, then $\text{res}_0 = \vec{a}_0 + \vec{b}_0$, $\text{res}_1 = \vec{a}_1 + \vec{b}_1$, ..., $\text{res}_{31} = \vec{a}_{31} + \vec{b}_{31}$.

Notice that all files must store **unsigned int** values in binary, have exactly the same nonzero size, and this size must be divisible by `BUFSIZE= 65536`. The program **sum** can be compiled with

```
clang -fopenmp sum.c -o sum
```

The usage is

```
./sum [number_of_files] [output_file_prefix] [input_file_prefix1] [input_file_prefix2] ...
```

3 Dependencies in POLYMULT

POLYMULT depends on several libraries and specifications that need to be present on your system prior to the installation. These libraries are:

1. **FLINT**, flintlib.org. Fast library for number theory;
2. **GMP**, gmplib.org. The GNU multiple precision arithmetic library;
3. **OpenMP**, openmp.org. The OpenMP API specification for parallel programming. Since version 4.2, every GCC compiler contains the implementation of the OpenMP specification.

Before installing POLYMULT, make sure that each of those libraries is installed.

4 Building and using POLYMULT

To compile POLYMULT use the following command:

```
gcc -fopenmp -lflint main.c init.c mult.c -o polymult
```

5 Reporting issues

The author wishes to be made aware of any bugs in the library or typos in this manual. Please send an email with your bug report to am3435@cornell.edu.

If possible please include details of your system, version of gcc, version of GMP and precise details of how to replicate the bug.

Note that POLYMULT needs to be linked against version 4.2.1 or later of GMP and must be compiled with gcc version 4.2 or later.

6 Files

The POLYMULT library consists of two parts:

1. The subroutines implemented in `init.c` initialize specific polynomials. See Subsection 7 for the descriptions of supported series, and Subsection 9 for the generic description of the contents of this file;
2. `mult.c` file contains the implementation of the out-of-core FFT-based multiplication technique of Hart, Tornara and Watkins, with OpenMP used for parallelization. See Subsection 10 for more details;
3. `main.c` contains the implementation of the command line program `polymult`, through which the multiplication is performed. See Subsection 11 for the instructions on how to run the program.

7 Supported series

For coprime integers a and m , with $0 \leq a \leq m$, define

$$\nabla_{a,m}(q) = \sum_{k=-\infty}^{+\infty} q^{k(mk+a)/2}$$

Many classical theta series can be expressed in terms of products of $cq^r \nabla_{a,m}(q^s)$. For example, the classical series

$$\theta_3(q) = \sum_{k=-\infty}^{+\infty} q^{k^2} = 1 + 2q + 2q^4 + 2q^9 + \dots \quad \text{and} \quad \nabla(q) = \sum_{k=0}^{+\infty} q^{k(k+1)/2} = 1 + q + q^3 + q^6 + q^{10} + \dots$$

used in [Mos14] can be expressed as

$$\theta_3(q) = \nabla_{0,1}(q^2) \quad \text{and} \quad \nabla(q) = \frac{1}{2} \nabla_{1,1}(q)$$

For a fixed positive integer n , POLYMULT calculates the following product:

$$c_0 q^{r_0} \nabla_{a_0, m_0}(q^{s_0}) \times \sum_{i=1}^n [c_{2i-1} q^{r_{2i-1}} \nabla_{a_{2i-1}, m_{2i-1}}(q^{s_{2i-1}}) \times c_{2i} q^{r_{2i}} \nabla_{a_{2i}, m_{2i}}(q^{s_{2i}})]$$

Notice that an ordered tuple (c, q, r, a, m) is admissible if and only if $\gcd(a, m) = 1$, $0 \leq a \leq m$, and $sk(mk + a)/2$ is a non-negative integer for every integer k . See Section 12 for example usage. **The values (c, q, r, a, m) must be specified in that particular order.**

IMPORTANT: The case $a = m = 1$ is special. If such parameters are provided, the program will generate $\frac{1}{2}cq^r \sum_{k=-\infty}^{+\infty} q^{sk(mk+a)/2}$, as opposed to $cq^r \sum_{k=-\infty}^{+\infty} q^{sk(mk+a)/2}$.

During the compilation, the following two macros may be defined through the `-D` command:

- **KEEP_FILES.** In order to save some space on the hard disk, the out-of-core polynomial multiplication routines delete intermediate files automatically. In order to keep any intermediate computations saved to hard disk, please define the `KEEP_FILES` macro. This may be useful for debugging purposes, or when the crashes on a server occur and you don't want your computations to get corrupted or lost. In this case, make sure that the prefixes of various collections of files differ from each other so that they won't get overwritten;
- **DEBUG.** Many subroutines of POLYMULT have several substeps. Such substeps are: initializing from or saving to files, bundling polynomials or reducing polynomials modulo a prime, restoring the coefficients using the Chinese Remainder Theorem, etc. By default, the timing and the status of those substeps do not get printed to the standard output. If you would like to see this information, for example in order to observe at which point of the execution your program crashes, please specify the `DEBUG` macro.

8 File types

The out-of-core routines produce many binary files on a hard disk. They may contain coefficients of a single polynomial either over the ring of integers, or reduced modulo several primes. The files are saved

in the form `prefix0`, `prefix1`, ..., `prefixM`, where `prefix` is the name of a particular collection of files and `M` is the last index. Note that in order for the program to work correctly, the total number of files `M + 1` must evenly divide the total number of coefficients stored in those files.

Let $n \in \mathbb{N}$, and consider a polynomial $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$ with $a_0, a_1, \dots, a_{n-1} \in \mathbb{N} \cup \{0\}$. Let $p_1, p_2, \dots, p_k$ be k distinct primes, and let m be a total number of files which evenly divides n . There are three types of collections of files that get produced:

- **Type 1.** These collections of files contain coefficients of $f(x)$ that are evenly distributed between each of m files:

```

prefix0:      a0,           a1,           ...   an/m-1
prefix1:      an/m,        an/m+1,        ...   a2n/m-1
...
prefixm-1:    a(m-1)n/m,    a(m-1)n/m+1,    ...   an-1

```

After the out-of-core multiplication and the restoration of coefficients, these collections of files contain the final result of the multiplication. It is also possible to convert the **Type 1** files into **Type 2** files in order to prepare the resulting polynomial for yet another out-of-core multiplication.

- **Type 2.** These collections of files contain coefficients of $F(x) = A_0 + A_1x + \dots + A_{n/B-1}x^{n/B-1}$, the *bundled* polynomial produced from $f(x)$ with some bundling parameter B which evenly divides n/m , and reduced modulo primes $p_1, p_2, \dots, p_k$:

```

prefix0:      A0 (mod p1),      A1 (mod p1),      ...   An/(mB)-1 (mod p1),
               A0 (mod p2),      A1 (mod p2),      ...   An/(mB)-1 (mod p2),
               ...
prefix1:      A0 (mod pk),      A1 (mod pk),      ...   An/(mB)-1 (mod pk)
               An/(mB) (mod p1),    An/(mB)+1 (mod p1), ...   A2n/(mB)-1 (mod p1),
               An/(mB) (mod p2),    An/(mB)+1 (mod p2), ...   A2n/(mB)-1 (mod p2),
               ...
               An/(mB) (mod pk),    An/(mB)+1 (mod pk), ...   A2n/(mB)-1 (mod pk)
...
prefixm-1:    A(m-1)n/(mB) (mod p1), A(m-1)n/(mB)+1 (mod p1), ...   An/B-1 (mod p1),
               A(m-1)n/(mB) (mod p2), A(m-1)n/(mB)+1 (mod p2), ...   An/B-1 (mod p2),
               ...
               A(m-1)n/(mB) (mod pk), A(m-1)n/(mB)+1 (mod pk), ...   An/B-1 (mod pk)

```

Before an out-of-core multiplication is performed, both polynomials get converted into collections of this type.

- **Type 3.** These collections of files contain coefficients of a polynomial that is a result of the out-of-core multiplication of two bundled polynomials modulo $p_1, p_2, \dots, p_k$. Though their structure is essentially the same as collections of **Type 2**, they do *not* contain coefficients of a bundled polynomial. Nevertheless, it is possible to restore the coefficients of a resulting polynomial distributed over files of **Type 1** from files of **Type 3**. This is done via the restoration algorithm described in Subsection 10.

9 Initialization

There are two initialization subroutines defined in `init.c`:

- The subroutine

```
void init_block_nabla(int * block, const ulong size, const ulong min,
const ulong c, const ulong r, const ulong s, const ulong a, const ulong m)
```

initializes a block of coefficients of a series $cq^r \nabla_{a,m}(q^s)$, starting from the coefficient `min` (included) and up to the coefficient `min + size` (excluded). In order for this subroutine to work properly, one must have $\gcd(a, m) = 1$, $0 \leq a \leq m$, and $sk(mk + a)/2$ an integer for every nonzero integer k . The case $a = m = 1$ is special. If such parameters are provided, the program will generate $\frac{1}{2}cq^r \sum_{k=-\infty}^{+\infty} q^{sk(mk+a)/2}$, as opposed to $cq^r \sum_{k=-\infty}^{+\infty} q^{sk(mk+a)/2}$.

- The subroutine

```
void init_block_nabla_product(int * block, const ulong size, const ulong min,
const ulong c1, const ulong r1, const ulong s1, const ulong a1, const ulong m1,
const ulong c2, const ulong r2, const ulong s2, const ulong a2, const ulong m2)
```

initializes a block of coefficients of a series $\prod_{i=1}^2 c_i q^{r_i} \nabla_{a_i, m_i}(q^{s_i})$, starting from the coefficient `min` (included) and up to the coefficient `min + size` (excluded). In order for this subroutine to work properly, one must have for both values of i that $\gcd(a_i, m_i) = 1$, $0 \leq a_i \leq m_i$, and $s_i k(m_i k + a_i)/2$ an integer for every nonzero integer k . The case $a_i = m_i = 1$ is special. If such parameters are provided, the program will generate $\frac{1}{2}c_i q^{r_i} \sum_{k=-\infty}^{+\infty} q^{s_i k(m_i k + a_i)/2}$, as opposed to $c_i q^{r_i} \sum_{k=-\infty}^{+\infty} q^{s_i k(m_i k + a_i)/2}$.

10 Out-of-core operations on polynomials

The following subroutines are defined in `mult.c`:

```
void init_primes(ulong * primes, const uint total_primes,
const ulong lowerbound)
```

Initializes a set of `total_primes` distinct primes, which immediately follow the `lowerbound`. This subroutine utilizes the `n_nextprime` function of FLINT, and is used in `multiply` and `divide` subroutines described below.

```
void init_files(const ulong * primes, const uint total_primes,
const ulong limit, const uint files, const uint bundle,
const uint bitsize, const int mod, const char * resultname,
const void * type, const char flags)
```

This subroutine “prepares” a polynomial for the out-of-core multiplication by initializing a **Type 2** collection of `files` binary files with a prefix `resultname`. The files contain coefficients of a *bundled* polynomial, produced from the first `limit` coefficients of a series called `type`, and reduced modulo `total_primes` primes specified in `primes`. If the coefficients of a series admit negative numbers and hence need to be initialized modulo a prime, this prime can be specified by a `mod` parameter (set `mod = 0` otherwise).

Note that the `type` parameter, instead of being a macro identifying a specific series, may be a prefix of a **Type 1** collection of `files` binary files (that is, a string of type `char *`). These files get deleted after the initialization of a polynomial.

The bundling parameter is `bundle`, and the bitsize parameter is `bitsize` (see [Mos14, Section 4.1]).

```
void ooc_multiply(const ulong * primes, const uint total_primes,
const ulong limit, const uint files, const uint bundle,
const char * resultname, const char * name1, const char * name2)
```

Performs an out-of-core multiplication of two *bundled* polynomials reduced modulo `total_primes` primes, specified in `primes`. Both polynomials are specified via **Type 2** collection of `files` binary files, and their prefixes are `name1` and `name2`. The result is saved into **Type 3** collection of `files` binary files with a prefix `resultname`. Both (non-bundled) polynomials have a degree `limit - 1`, and the `limit` parameter has to be evenly divisible by `files`. The bundling parameter is `bundle`. The resulting polynomial contains the information only on the first `limit` coefficients.

```
void square(const ulong * primes, const uint total_primes,
            const ulong limit, const uint files, const uint bundle,
            const char * resultname, const char * name1)
```

Performs an out-of-core squaring of a *bundled* polynomial reduced modulo `total_primes` primes, specified in `primes`. A polynomial with `limit` coefficients is specified via a **Type 2** collection of `files` binary files, and its prefix is `name1`. The `limit` parameter has to be evenly divisible by `files`. The result is saved into a **Type 3** collection of `files` binary files with a prefix `resultname`. The bundling parameter is `bundle`. Note that the squaring gets performed *completely*; that is, no truncation occurs, and the resulting files actually contain information on `2limit - 1` coefficients.

```
void restore_coeff(const ulong * primes, const uint total_primes,
                  const ulong limit, const uint files, const uint bundle,
                  const uint bitsize, const int mod, const char * resultname,
                  const char * name1, const int flags)
```

Restores `limit` coefficients from a **Type 3** collection of `files` binary files with a prefix `name1`, which contain the result of multiplication of two bundled polynomials reduced modulo `total_primes` primes, specified in `primes`. The result is saved into a **Type 1** collection of `files` binary files with a prefix `resultname`. The `limit` parameter has to be evenly divisible by `files`. The bundling parameter is `bundle` and the `bitsize` parameter is `bitsize`. If the restored coefficients need to be reduced modulo a prime, this prime can be specified in `mod` (set `mod = 0` otherwise).

```
void multiply(const ulong limit, const uint files, const uint bundle,
             const uint bound, const char * resultname, const char * folder,
             const char type1, const char type2)
```

Performs the out-of-core multiplication of two polynomials of types `type1` and `type2`. The result is saved into **Type 1** collection of `files` binary files with a prefix `resultname` in a folder `folder`. The number of coefficients of both polynomials is `limit`, and the upper bound on the coefficients of the resulting polynomial is `bound`. The bundling parameter is `bundle`.

11 Utilizing the executable

Recall that POLYMULT calculates the following product, truncating it to some fixed degree:

$$c_0 q^{r_0} \nabla_{a_0, m_0}(q^{s_0}) \times \sum_{i=1}^n [c_{2i-1} q^{r_{2i-1}} \nabla_{a_{2i-1}, m_{2i-1}}(q^{s_{2i-1}}) \times c_{2i} q^{r_{2i}} \nabla_{a_{2i}, m_{2i}}(q^{s_{2i}})]$$

The `polymult` executable admits at least 21 parameters. If the total number of parameters is less than 21 or it is not equal to 1 modulo 10, the following prompt comes out:

```
./polymult [poly_degree] [number_of_files] [bundle] [bound] [resultname] [folder]
[c0] [r0] [s0] [a0] [m0] [c1] [r1] [s1] [a1] [m1] ...
```


The parameters provided are:

- `[poly_degree]`: degree to which polynomials get multiplied or divided;
- `[number_of_files]`: number of files in a single collection to which the coefficients of a resulting polynomial, as well as intermediate computations, will be saved;
- `[bundle]`: bundling parameter;
- `[bound]`: upper-bound on the coefficients of a resulting polynomial;
- `[resultname]`: prefix of files where the result get saved;
- `[folder]`: folder;
- `[c0] [r0] [s0] [a0] [m0]` correspond to $c_0 q^{r_0} \nabla_{a_0, m_0}(q^{s_0})$;
- `[c1] [r1] [s1] [a1] [m1] ...` correspond to $[c_{2i-1} q^{r_{2i-1}} \nabla_{a_{2i-1}, m_{2i-1}}(q^{s_{2i-1}}) \times c_{2i} q^{r_{2i}} \nabla_{a_{2i}, m_{2i}}(q^{s_{2i}})]$ for $i = 1, 2, \dots$. You can provide as many of these parameters as you like, but their total number must be divisible by 10.

See Subsection 12 for some examples on how to run the program.

12 Examples

Example 1. The following command tabulates all Hurwitz class numbers $H(\Delta)$ of imaginary quadratic fields to $|\Delta| < 15 \cdot 2^{28}$, where $|\Delta| \equiv 8 \pmod{16}$. In particular, it multiplies

$$\theta_3(q) = \nabla_{0,1}(q^2) \quad \text{and} \quad [\nabla(q^2)]^2 = \left[\frac{1}{2} \nabla_{1,1}(q^2) \right]^2$$

to degree $15 \cdot 2^{24} = 251658240$ out-of-core, with the bundling parameter $2^8 = 256$ and the upper bound on the coefficients of $\theta_3(q) \nabla^2(q^2)$ given by 1000000. The coefficients of $\sum_{k=0}^{\infty} H(16k+8)q^k = \theta_3(q) \nabla^2(q^2)$ get saved into 32 files: `/home/h8mod16.0`, `/home/h8mod16.1`, ..., `/home/h8mod16.31`.

```
./polymult 251658240 32 256 1000000 h8mod16. /home 1 0 2 0 1 1 0 2 1 1 1 0 2 1 1
```

Example 2. The following command tabulates all Hurwitz class numbers $H(\Delta)$ (multiplied by 2) of imaginary quadratic fields to $|\Delta| < 15 \cdot 2^{28}$, where $|\Delta| \equiv 4 \pmod{16}$. In particular, it multiplies

$$\nabla(q^2) = \frac{1}{2} \nabla_{1,1}(q^2) \quad \text{and} \quad [\theta_3(q)]^2 = [\nabla_{0,1}(q^2)]^2$$

to degree $15 \cdot 2^{24} = 251658240$ out-of-core, with the bundling parameter $2^8 = 256$ and the upper bound on the coefficients of $\nabla(q^2) \theta_3^2(q)$ given by 1000000. The coefficients of $2 \sum_{k=0}^{\infty} H(16k+4)q^k = \nabla(q^2) \theta_3^2(q)$ get saved into 32 files: `/home/h4mod16.0`, `/home/h4mod16.1`, ..., `/home/h4mod16.31`.

```
./polymult 251658240 32 256 1000000 h4mod16. /home 1 0 2 1 1 1 0 2 0 1 1 0 2 0 1
```

Example 3. The following command tabulates all Hurwitz class numbers $H(\Delta)$ (multiplied by 3) of imaginary quadratic fields to $|\Delta| < 15 \cdot 2^{28}$, where $|\Delta| \equiv 3 \pmod{8}$. In particular, it multiplies

$$\nabla(q) = \frac{1}{2}\nabla_{1,1}(q) \quad \text{and} \quad [\nabla(q)]^2 = \left[\frac{1}{2}\nabla_{1,1}(q)\right]^2$$

to degree $15 \cdot 2^{25} = 503316480$ out-of-core, with the bundling parameter $2^8 = 256$ and the upper bound on the coefficients of $3 \sum_{k=0}^{\infty} H(8k+3)q^k = \nabla^3(q)$ given by 1000000. The coefficients of $\nabla^3(q)$ get saved into 32 files: `/home/h3mod8.0`, `/home/h3mod8.1`, $\dots$, `/home/h3mod8.31`.

```
./polymult 503316480 32 256 1000000 h3mod8. /home 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1
```

Example 4. The following command tabulates all Hurwitz class numbers $H(\Delta)$ (multiplied by 2) of imaginary quadratic fields to $|\Delta| < 15 \cdot 2^{28}$, where $|\Delta| \equiv 7 \pmod{24}$. In particular, it multiplies

$$\nabla(q) \quad \text{and} \quad [\nabla_{1,3}(q)\nabla(q) + \nabla_{1,3}(q^4)\nabla_{0,1}(q^4) + 2q\nabla_{2,3}(q^4)\nabla(q^4)]$$

to degree $5 \cdot 2^{25} = 167772160$ out-of-core, with the bundling parameter $2^8 = 256$ and the upper bound on the coefficients of $2 \sum_{k=0}^{\infty} H(24k+7)q^k$ given by 1000000. The coefficients get saved into 32 files: `/home/h7mod24.0`, `/home/h7mod24.1`, $\dots$, `/home/h7mod24.31`.

```
./polymult 167772160 32 256 1000000 h7mod24. /home
1 0 1 1 1 1 0 1 1 3 1 0 1 1 1 1 0 4 1 3 1 0 4 0 1 2 1 4 2 3 1 0 4 1 1
```

Example 5. The following command tabulates all Hurwitz class numbers $H(\Delta)$ (multiplied by 2) of imaginary quadratic fields to $|\Delta| < 15 \cdot 2^{28}$, where $|\Delta| \equiv 15 \pmod{24}$. In particular, it multiplies

$$\nabla(q) \quad \text{and} \quad [\nabla(q^3)\nabla(q) + q\nabla(q^{12})\nabla_{0,1}(q^4) + \nabla_{0,1}(q^{12})\nabla(q^4)]$$

to degree $5 \cdot 2^{25} = 167772160$ out-of-core, with the bundling parameter $2^8 = 256$ and the upper bound on the coefficients of $\sum_{k=0}^{\infty} H(24k+15)q^k$ given by 1000000. The coefficients get saved into 32 files: `/home/h15mod24.0`, `/home/h15mod24.1`, $\dots$, `/home/h15mod24.31`.

```
./polymult 167772160 32 256 1000000 h7mod24. /home
1 0 1 1 1 1 0 3 1 1 1 0 1 1 1 1 1 12 1 1 1 0 4 0 1 1 0 12 0 1 1 0 4 1 1
```

13 Utilizing the library

In order to utilize the library, place the files `init.h` and `mult.h` into your include path, and copy the library `libpolymult.a` into your library path. By writing

```
#include <mult.h>
```

among other inclusions in your file you will gain access to all the subroutines defined in `mult.h`. Same applies to `init.h`.

When compiling, link the library to your program by writing `-lpolymult`. Don't forget to link all the other libraries that POLYMULT depends on (see Subsection 3 for the complete list).

References

- [BK22] K. Bringmann and B. Kane, *Class numbers and representations by ternary quadratic forms with congruence conditions*, Math. Comp. 91 (333), pp. 295–329, 2022.
- [GG03] J. von zur Gathen, J. Gerhard, *Modern Computer Algebra*, Cambridge University Press, 2nd edition, 2003.
- [HTW10] W. B. Hart, G. Tornara, M. Watkins, *Congruent number theta coefficients to 10^{12}* , Algorithmic Number Theory — ANTS-IX (Nancy, France), Lecture Notes in Computer Science 6197, Springer-Verlag, Berlin, pp. 186 – 200, 2010.
- [Mos14] A. S. Mosunov, *Unconditional Class Group Tabulation to 2^{40}* , Master’s thesis, University of Calgary, Calgary, Alberta, 2014.